Batch

19/4/26 15:18

AWS Batch

- Fully managed service that enables you to run batch computing workloads on the AWS Cloud.
- It dynamically provisions the optimal quantity and compute resources (e.g., CPU or memory-optimized instances) based on the volume and specific resource requirements of the batch jobs submitted.
- It eliminates the need to install and manage batch computing software or server clusters.
- It is a regional service that simplifies running batch jobs across multiple AZs within a region.

Features

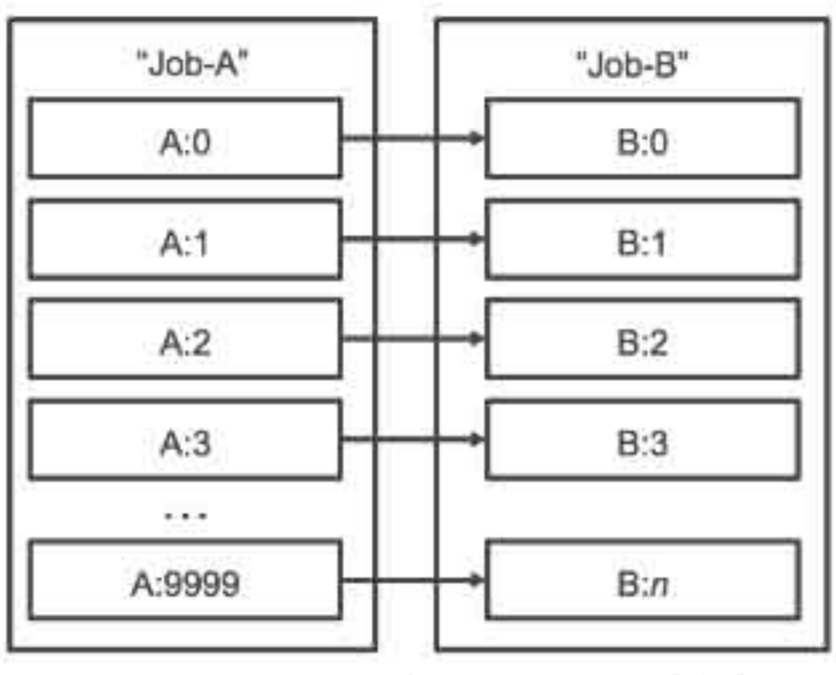
- Batch chooses where to run the jobs, automatically launching additional AWS capacity if needed. It dynamically provisions the optimal quantity and type of compute resources (CPU, memory-optimized, etc.) based on your jobs' requirements.
- Manages compute environments and job queues, allowing you to efficiently run thousands of jobs using Amazon EC2, EC2 Spot Instances, AWS Fargate, or Amazon EKS.
- Provides the ability to submit jobs that are part of a pipeline or workflow, enabling you to express interdependencies (e.g., Job B starts only after Job A succeeds).
- Supports scheduling policies that ensure compute resources are allocated fairly among users or workloads (via shared identifiers), preventing a single user from monopolizing the cluster.
- Enables you to run single large-scale, tightly coupled, high-performance computing (HPC) applications that span multiple EC2 instances.
- Batch carefully monitors the progress of your jobs. When capacity is no longer needed, it is automatically removed to minimize costs.

Components

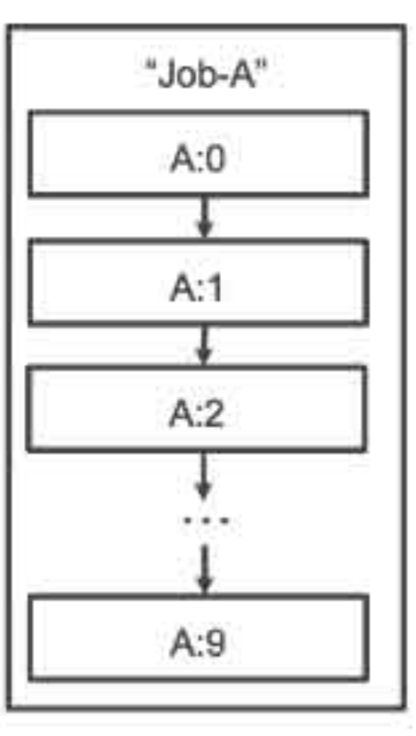
- Jobs
- Job Definitions
- Job Queues
- Compute Environment

Jobs

- A unit of work (such as a shell script, a Linux executable, or a Docker container image) that you submit to Batch.
- Jobs can reference other jobs by name or by ID, and can be dependent on the successful completion of other jobs.
- Job types
  - **Single** for single Job
    - Runs on a single EC2 instance or Fargate task.
  - **Array** for array Job of size 2 to 10,000
- An **array job** shares common job parameters, such as the job definition, vCPUs, and memory. It runs as a collection of related, yet separate, basic jobs that may be distributed across multiple hosts and may run concurrently.
- **Multi-node parallel jobs** enable you to run single large-scale, tightly coupled, high-performance computing applications and distributed GPU model training jobs that span multiple Amazon EC2 instances.
  - Batch lets you specify up to five distinct node groups for each job. Each group can have its own container images, commands, environment variables, and so on.
  - Each multi-node parallel job contains a *main node*, which is launched first. After the main node is up, the *child nodes* are launched and started. If the main node exits, the job is considered finished, and the child nodes are stopped.
  - Not supported on computing environments that use Spot Instances.
- Dependencies
  - A job may have up to 20 dependencies.
  - For **Job depends on**, enter the job IDs for any jobs that must finish before this job starts.
  - (Array jobs only) For **N-To-N job dependencies**, specify one or more job IDs for any array of jobs for which each child job index of this job should depend on the corresponding child index job of the dependency.



  - (Array jobs only) **Run children sequentially** creates a SEQUENTIAL dependency for the current array job. This ensures that each child index job waits for its earlier sibling to finish.



- States
  - SUBMITTED – a job that has been submitted to the queue, and has not yet been evaluated by the scheduler.
  - PENDING – a job that resides in the queue and is not yet able to run due to a dependency on another job or resource.
  - RUNNABLE – a job that resides in the queue has no outstanding dependencies and is, therefore, ready to be scheduled to a host. Jobs in this state are started as soon as sufficient resources are available in one of the computing environments that are mapped to the job's queue.
  - STARTING – jobs have been scheduled to a host and the relevant container initiation operations are underway.
  - RUNNING – the job is running as a container job on an Amazon ECS container instance within a computing environment. When the job's container exits, the process exit code determines whether the job succeeded or failed. An exit code of 0 indicates success, and any non-zero exit code indicates failure.
  - SUCCEEDED – the job has successfully completed with an exit code of 0. The job state for SUCCEEDED jobs is persisted for 24 hours.
  - FAILED – the job has failed all available attempts. The job state for FAILED jobs is persisted for 24 hours.
- Scheduling Policy Support: Jobs can now have "Share Identifier" tag to participate in fair-share scheduling.
- You can apply a retry strategy to your jobs and job definitions that allows failed jobs to be automatically retried.
- You can configure a timeout duration for your jobs so that if a job runs longer than that, Batch terminates the job. If a job is terminated for exceeding the timeout duration, it is not retried. If it fails on its own, then it can retry if retries are enabled, and the timeout countdown is started over for the new attempt.

Job Definitions

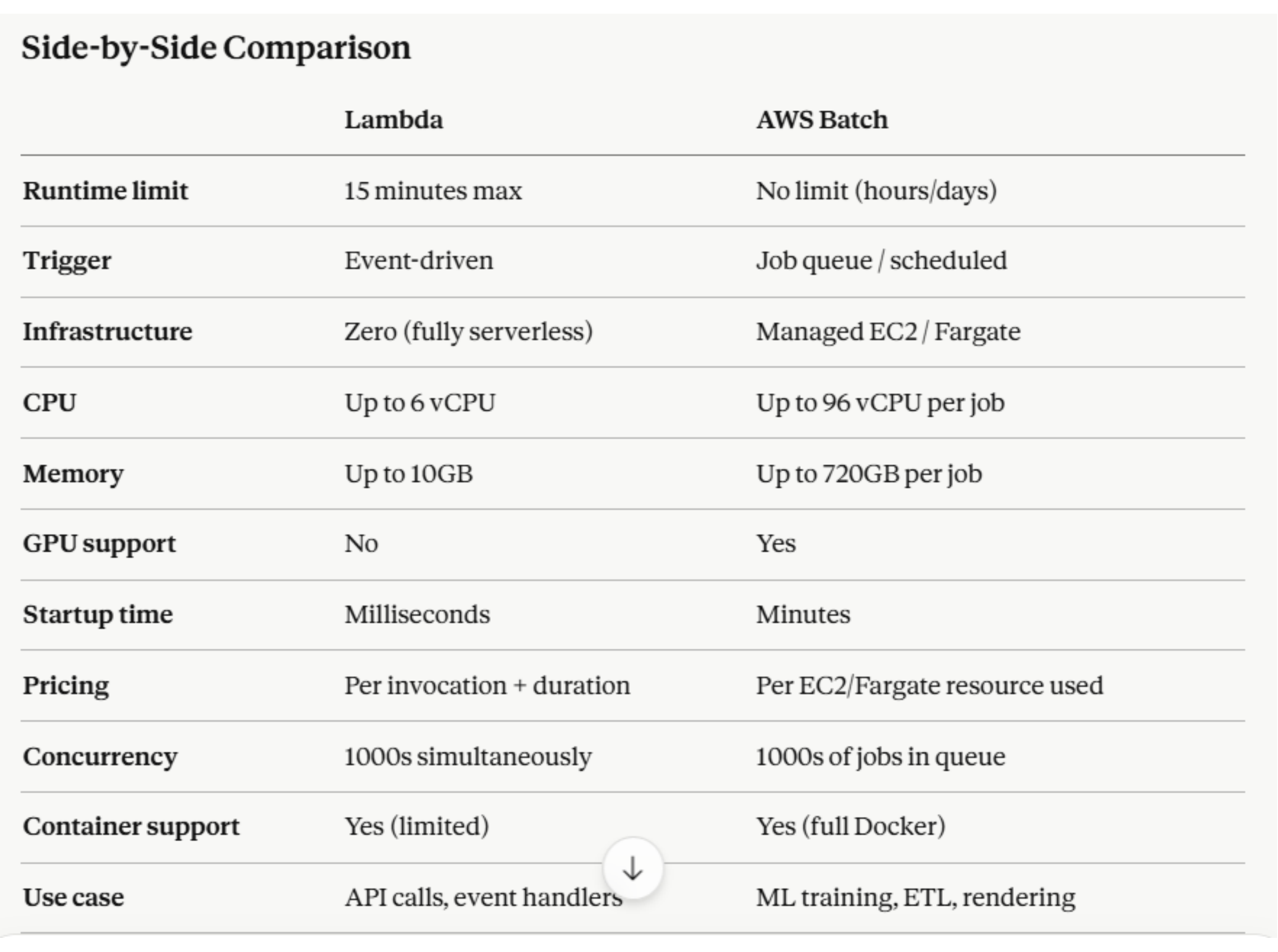
- Specifies how jobs are to be run.
- The definition can contain:
  - an IAM role to provide programmatic access to other AWS resources.
  - memory and CPU requirements for the job.
  - controls for container properties, environment variables, and mount points for persistent storage.

Job Queues

- This is where a Batch job resides until it is scheduled onto a computing environment.
- You can associate one or more compute environments with a job queue.
- You can assign priority values for the compute environments and even across job queues themselves.
- The **Batch Scheduler** evaluates when, where, and how to run jobs that have been submitted to a job queue. Jobs run in approximately the order in which they are submitted as long as all dependencies on other jobs have been met.
- You can now attach a "Scheduling Policy" to a job queue, enabling **Fair Share Scheduling**, which allocates compute resources based on weights assigned to different users or projects known as "share identifiers."

AWS Batch Compute Environment

  - A set of managed or unmanaged compute resources (such as EC2 instances) that are used to run jobs.
  - Compute environments contain the Amazon ECS container instances that are used to run containerized batch jobs.
  - A given compute environment can be mapped to one or many job queues.
- **Managed Compute Environment**
  - Batch manages the capacity and instance types of the compute resources within the environment, based on the compute resource specification that you define when you create the computing environment.
  - You can choose to use EC2 On-Demand Instances or Spot Instances in your managed compute environment.
  - ECS container instances are launched into the VPC and subnets that you specify when you create the computing environment.
- **Unmanaged Compute Environment**
  - You manage your own computing resources in this environment.
  - After you have created your unmanaged compute environment, use the *DescribeComputeEnvironments* API operation to view the compute environment details and find the ECS cluster that is associated with the environment, and manually launch your container instances into that cluster.



Side-by-Side Comparison

	Lambda	AWS Batch
Runtime limit	15 minutes max	No limit (hours/days)
Trigger	Event-driven	Job queue / scheduled
Infrastructure	Zero (fully serverless)	Managed EC2 / Fargate
CPU	Up to 6 vCPU	Up to 96 vCPU per job
Memory	Up to 10GB	Up to 720GB per job
GPU support	No	Yes
Startup time	Milliseconds	Minutes
Pricing	Per invocation + duration	Per EC2/Fargate resource used
Concurrency	1000s simultaneously	1000s of jobs in queue
Container support	Yes (limited)	Yes (full Docker)
Use case	API calls, event handlers	ML training, ETL, rendering

AWS Batch Security

- By default, IAM users don't have permission to create or modify AWS Batch resources or perform tasks using the AWS Batch API.
- Take advantage of IAM policies, roles, and permissions.
- You can control access to AWS Batch resources based on the tag values.
- Service-Linked Role: AWS Batch uses a service-linked role (AWSServiceRoleForBatch) to call other AWS services (like EC2, ECS, EKS) on your behalf.
- EKS Auth: When using EKS compute environments, Batch manages the aws-auth ConfigMap to grant permissions to the nodes it provisions.

AWS Batch Monitoring

- You can use the **AWS Batch event stream for CloudWatch Events** to receive near real-time notifications regarding the current state of jobs that have been submitted to your job queues.
- Events from the AWS Batch event stream are ensured to be delivered at least one time.
- CloudTrail captures all API calls for AWS Batch as events.

AWS Batch Pricing

- There is no additional charge for AWS Batch. You pay for resources you create to store and run your application.

From <https://tutorialsdojo.com/aws-batch/>